

TITLE

Rocket Fin Bending Vibration Analysis

Objective: Evaluate the dynamic response of a rocket fin subjected to aerodynamic and propulsion-induced excitation.

Methodology: In this code, a fin has been modelled as a cantilevered beam using Euler-Bernoulli beam theory. Modal analysis was performed to determine the natural frequencies and mode shapes of the structure. The dynamic behaviour of the fin was further investigated through frequency response analysis and damped transient vibration simulations to evaluate resonance characteristics under aerodynamic and propulsion-induced vibration.

Software used are Python, NumPy, SciPy and Matplotlib.

Fin Properties:

- Length: 150.0 mm
- Chord: 80.0 mm
- Thickness: 3.0 mm
- Material: Aluminum ($\rho = 2700 \text{ kg/m}^3$, $E = 70 \text{ GPa}$)
- Second moment I: $1.800\text{e-}10 \text{ m}^4$
- Mass per length: 0.648 kg/m

Governing Equation:

The code is based on Euler-Bernoulli beam theory:

$$EI \frac{\partial^4 w}{\partial x^4} + \rho A \frac{\partial^2 w}{\partial t^2} = 0$$

Where the rocket fin was approximated as a uniform cantilever beam and analysed using the aforementioned formula.

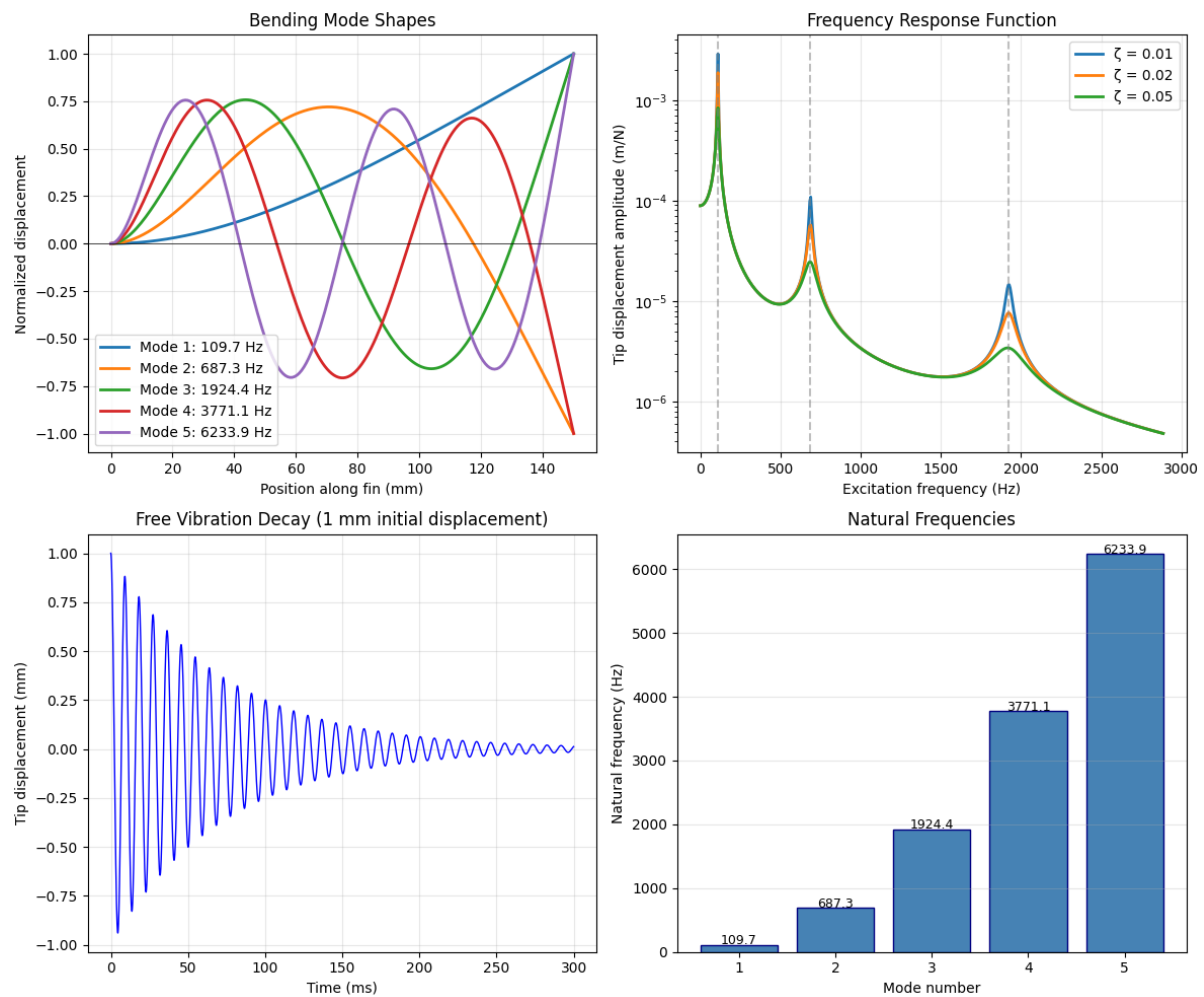
Here, $w(x,t)$ = Lateral deflection of the beam as a function of position and time,

EI = Flexural rigidity of the beam (Young's modulus x second moment of area),

A = Area of cross section of the beam,

ρ = Density of the beam.

Results:



- **Mode Shapes:** The first five bending modes were successfully extracted, with the fundamental bending mode occurring at 109.67 Hz and higher modes occurring at progressively larger frequencies.
- **FRF:** Frequency response function showing resonance amplification and damping effects under harmonic excitation.
- **Transient Response:** Time-domain damped free vibration response lasting for a duration of 300 milliseconds following an imposed maximum initial tip displacement of 1mm.
- **Natural Frequencies:** Calculated bending eigenfrequencies used to assess resonance susceptibility and structural dynamic performance.

Engineering Findings:

- First natural frequency = 109.67Hz
- Forced response identified at 98.7Hz, close to the first natural frequency of 109.67 Hz, demonstrating the sensitivity of the structure to resonance excitation.
- Maximum tip displacement = 4.508mm
- The first bending mode dominated the dynamic response, while higher-order modes exhibited reduced contribution to overall tip displacement under typical vibration conditions.

Conclusion:

- Developed a computational structural dynamics model to analyze bending vibration characteristics of rocket fin structures using Python, NumPy, SciPy, and Matplotlib.
- Implemented cantilever beam modal analysis to calculate natural frequencies, mode shapes, transient response, and forced harmonic vibration behavior.
- Simulated damped and forced vibrational response using modal superposition techniques and frequency response functions relevant to aerospace structural resonance analysis.
- Generated engineering visualizations including mode shapes, frequency response plots, transient vibration decay, and resonance characteristics for structural dynamic assessment.

Engineering Significance:

Rocket fins are subjected to aerodynamic forcing, engine-induced vibration, and transient loading throughout flight. If excitation frequencies approach the natural frequencies of the fin, resonance may result in excessive vibration amplitudes and potential structural failure. The present study demonstrates how modal analysis and frequency response techniques can be used during preliminary design stages to identify critical frequencies and evaluate the effectiveness of structural damping.

Future Work:

- Validation using FEA modal analysis in ANSYS.
- Investigation of carbon-fibre reinforced composite fins under similar loading conditions.
- Comparative study of Aluminium and Composite Fin configurations dynamic performance.
- Investigation of the interaction of aeroelastic coupling with aerodynamic loading.
- Structural optimization for minimum mass and maximum stiffness.
- Extension to transient launch and propulsion-induced loading environments

Key skills demonstrated: Structural dynamics, modal analysis, numerical methods, scientific computing (Python), aerospace structures, vibration analysis, engineering data visualization.

The code:

```
import numpy as np
import matplotlib.pyplot as plt
from dataclasses import dataclass
from typing import Tuple
```

```
@dataclass
```

```
class FinProperties:
```

```
    """Material and geometric properties of the rocket fin."""
```

```

length: float      # Fin span (root to tip) [m]
chord: float       # Average chord length [m]
thickness: float   # Fin thickness [m]
density: float     # Material density [kg/m³]
elastic_modulus: float # Young's modulus [Pa]

```

```

@property
def area(self) -> float:
    """Cross-sectional area [m²]."""
    return self.chord * self.thickness

```

```

@property
def second_moment(self) -> float:
    """Second moment of area for bending about chord axis [m⁴]."""
    return (self.chord * self.thickness**3) / 12

```

```

@property
def mass_per_length(self) -> float:
    """Linear mass density [kg/m]."""
    return self.density * self.area

```

```

def cantilever_eigenvalues(n_modes: int = 5) -> np.ndarray:
    """
    Solve the characteristic equation for cantilever beam eigenvalues.

```

The equation is: $\cos(\beta L) * \cosh(\beta L) + 1 = 0$

Returns the first n_modes values of βL .

```

    """

```

```

from scipy.optimize import brentq

```

```

def characteristic_eq(beta_L: float) -> float:
    return np.cos(beta_L) * np.cosh(beta_L) + 1

```

```

eigenvalues = []
# Search intervals where roots exist (between asymptotes of cos)
search_starts = [(1.5 + i * np.pi) for i in range(n_modes)]

```

```

for start in search_starts:
    try:
        root = brentq(characteristic_eq, start, start + np.pi - 0.1)
        eigenvalues.append(root)

```

```

except ValueError:
    continue

return np.array(eigenvalues[:n_modes])

def natural_frequencies(fin: FinProperties, n_modes: int = 5) -> np.ndarray:
    """
    Calculate natural frequencies of bending vibration.

    For a cantilever beam:
     $\omega_n = (\beta_n * L)^2 * \sqrt{EI / (\rho A * L^4)}$ 

    Returns frequencies in Hz.
    """
    beta_L = cantilever_eigenvalues(n_modes)

    EI = fin.elastic_modulus * fin.second_moment
    rhoA = fin.mass_per_length
    L = fin.length

    omega = (beta_L**2) * np.sqrt(EI / (rhoA * L**4))
    frequencies_hz = omega / (2 * np.pi)

    return frequencies_hz

def mode_shape(x: np.ndarray, beta_L: float, L: float) -> np.ndarray:
    """
    Calculate the normalized mode shape for cantilever bending.

     $\phi(x) = \cosh(\beta x) - \cos(\beta x) - \sigma[\sinh(\beta x) - \sin(\beta x)]$ 

    where  $\sigma = (\cosh(\beta L) + \cos(\beta L)) / (\sinh(\beta L) + \sin(\beta L))$ 
    """
    beta = beta_L / L

    sigma = (np.cosh(beta_L) + np.cos(beta_L)) / (np.sinh(beta_L) + np.sin(beta_L))

    phi = (np.cosh(beta * x) - np.cos(beta * x)
           - sigma * (np.sinh(beta * x) - np.sin(beta * x)))

    # Normalize so max displacement = 1

```

```
return phi / np.max(np.abs(phi))
```

```
def forced_response(
    fin: FinProperties,
    force_amplitude: float,
    force_frequency: float,
    force_location: float,
    damping_ratio: float = 0.02,
    n_modes: int = 5
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Calculate steady-state forced vibration response using modal superposition.

    Args:
        fin: Fin properties
        force_amplitude: Peak force [N]
        force_frequency: Excitation frequency [Hz]
        force_location: Distance from root where force is applied [m]
        damping_ratio: Modal damping ratio (typical: 0.01-0.05)
        n_modes: Number of modes to include

    Returns:
        x: Position along fin [m]
        w_amplitude: Displacement amplitude at each position [m]
    """
    x = np.linspace(0, fin.length, 200)
    omega = 2 * np.pi * force_frequency

    freqs = natural_frequencies(fin, n_modes)
    beta_L_values = cantilever_eigenvalues(n_modes)

    w_amplitude = np.zeros_like(x)

    for i, (f_n, beta_L) in enumerate(zip(freqs, beta_L_values)):
        omega_n = 2 * np.pi * f_n

        # Mode shape
        phi = mode_shape(x, beta_L, fin.length)
        phi_force = mode_shape(np.array([force_location]), beta_L, fin.length)[0]

        # Modal mass (for normalized mode shape, approximate)
        modal_mass = fin.mass_per_length * fin.length * 0.25 # Approximate
```

```

# Frequency response function (magnitude)
r = omega / omega_n
H = 1 / np.sqrt((1 - r**2)**2 + (2 * damping_ratio * r)**2)

# Modal contribution
w_amplitude += (force_amplitude * phi_force * phi * H) / (modal_mass * omega_n**2)

return x, np.abs(w_amplitude)

def transient_response(
    fin: FinProperties,
    initial_tip_displacement: float,
    damping_ratio: float = 0.02,
    t_max: float = 0.5,
    n_modes: int = 3
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Calculate transient free vibration from initial displacement.

    Returns time history of tip displacement.
    """
    dt = 1e-5
    t = np.arange(0, t_max, dt)

    freqs = natural_frequencies(fin, n_modes)
    beta_L_values = cantilever_eigenvalues(n_modes)

    # Get mode shapes at tip
    phi_tip = np.array([mode_shape(np.array([fin.length]), bl, fin.length)[0]
                        for bl in beta_L_values])

    # Initial modal coordinates (simplified - assumes first mode dominates)
    q0 = np.zeros(n_modes)
    q0[0] = initial_tip_displacement / phi_tip[0]

    w_tip = np.zeros_like(t)

    for i, (f_n, phi_t) in enumerate(zip(freqs, phi_tip)):
        omega_n = 2 * np.pi * f_n
        omega_d = omega_n * np.sqrt(1 - damping_ratio**2) # Damped frequency

```

```

# Free vibration response
w_tip += (q0[i] * phi_t * np.exp(-damping_ratio * omega_n * t)
          * np.cos(omega_d * t))

return t, w_tip

def plot_analysis(fin: FinProperties):
    """Generate comprehensive visualization of fin vibration characteristics."""
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    # 1. Mode shapes
    ax1 = axes[0, 0]
    x = np.linspace(0, fin.length, 200)
    beta_L_values = cantilever_eigenvalues(5)
    freqs = natural_frequencies(fin, 5)

    for i, (beta_L, f) in enumerate(zip(beta_L_values, freqs)):
        phi = mode_shape(x, beta_L, fin.length)
        ax1.plot(x * 1000, phi, label=f'Mode {i+1}: {f:.1f} Hz', linewidth=2)

    ax1.set_xlabel('Position along fin (mm)')
    ax1.set_ylabel('Normalized displacement')
    ax1.set_title('Bending Mode Shapes')
    ax1.legend()
    ax1.grid(True, alpha=0.3)
    ax1.axhline(y=0, color='k', linewidth=0.5)

    # 2. Frequency response
    ax2 = axes[0, 1]
    freq_range = np.linspace(1, freqs[2] * 1.5, 500)

    for zeta in [0.01, 0.02, 0.05]:
        response = []
        for f in freq_range:
            _, w = forced_response(fin, 1.0, f, fin.length, zeta, n_modes=3)
            response.append(np.max(w))
        ax2.semilogy(freq_range, response, label=f' $\zeta = {zeta}$ ', linewidth=2)

    for f in freqs[:3]:
        ax2.axvline(x=f, color='gray', linestyle='--', alpha=0.5)

    ax2.set_xlabel('Excitation frequency (Hz)')

```



```

ax2.set_ylabel('Tip displacement amplitude (m/N)')
ax2.set_title('Frequency Response Function')
ax2.legend()
ax2.grid(True, alpha=0.3)

# 3. Transient response
ax3 = axes[1, 0]
t, w = transient_response(fin, 0.001, damping_ratio=0.02, t_max=0.3)
ax3.plot(t * 1000, w * 1000, 'b-', linewidth=1)
ax3.set_xlabel('Time (ms)')
ax3.set_ylabel('Tip displacement (mm)')
ax3.set_title('Free Vibration Decay (1 mm initial displacement)')
ax3.grid(True, alpha=0.3)

# 4. Natural frequencies bar chart
ax4 = axes[1, 1]
modes = range(1, 6)
ax4.bar(modes, freqs, color='steelblue', edgecolor='navy')
ax4.set_xlabel('Mode number')
ax4.set_ylabel('Natural frequency (Hz)')
ax4.set_title('Natural Frequencies')
ax4.grid(True, alpha=0.3, axis='y')

for i, f in enumerate(freqs):
    ax4.text(i + 1, f + freqs[0] * 0.05, f'{f:.1f}', ha='center', fontsize=9)

plt.tight_layout()
plt.savefig('fin_vibration_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

# Example usage
if __name__ == "__main__":
    # Typical rocket fin (aluminum)
    fin = FinProperties(
        length=0.15,      # 150 mm span
        chord=0.08,       # 80 mm chord
        thickness=0.003,   # 3 mm thick
        density=2700,      # Aluminum
        elastic_modulus=70e9 # 70 GPa
    )

    print("=" * 50)

```

```

print("ROCKET FIN BENDING VIBRATION ANALYSIS")
print("=" * 50)
print(f"\nFin Properties:")
print(f" Length:      {fin.length * 1000:.1f} mm")
print(f" Chord:        {fin.chord * 1000:.1f} mm")
print(f" Thickness:     {fin.thickness * 1000:.1f} mm")
print(f" Material:      Aluminum ( $\rho = \{fin.density\}$  kg/m3, E =
{fin.elastic_modulus/1e9:.0f} GPa)")
print(f" Second moment I: {fin.second_moment:.3e} m4")
print(f" Mass per length: {fin.mass_per_length:.3f} kg/m")

# Calculate natural frequencies
freqs = natural_frequencies(fin, n_modes=5)
print(f"\nNatural Frequencies (Bending):")
for i, f in enumerate(freqs):
    print(f" Mode {i+1}: {f:8.2f} Hz ({f*60:.0f} CPM)")

# Forced response at specific frequency
test_freq = freqs[0] * 0.9 # Near first resonance
x, w = forced_response(fin, force_amplitude=10, force_frequency=test_freq,
    force_location=fin.length, damping_ratio=0.02)
print(f"\nForced Response at {test_freq:.1f} Hz (10 N tip load):")
print(f" Max tip displacement: {np.max(w)*1000:.3f} mm")

# Generate plots
plot_analysis(fin)

```